torch.asarray#

https://pytorch.org/docs/stable/generated/torch.asarray.html#torch.asarray

Description:

Converts obj to a tensor. a NumPy array or a NumPy scalar

Function Signature

```
torch.asarray(obj: Any, *, dtype: Optional[dtype], device:
Optional[DeviceLikeType], copy: Optional[bool] = None,
requires_grad: bool = False) → Tensor#

Parameters

Keyword Arguments
```

Parameters

Keyword

dtype (torch.dtype, optional) – the datatype of the returned tensor. Default: None, which causes the datatype of the returned tensor to be inferred from obj. copy (bool, optional) – controls whether the returned tensor shares memory with obj. Default: None, which causes the returned tensor to share memory with obj whenever possible. If True then the returned tensor does not share its memory. If False then the returned tensor shares its memory with obj and an error is thrown if it cannot. device (torch.device, optional) – the device of the returned tensor. Default: None, which causes the device of obj to be used. Or, if obj is a Python sequence, the current default device will be used. requires_grad (bool, optional) – whether the returned tensor requires grad. Default: False, which causes t

Code Examples

```
>>> a = torch.tensor([1, 2, 3])
>>> # Shares memory with tensor 'a'
>>> b = torch.asarray(a)
>>> a.data_ptr() == b.data_ptr()
True
>>> # Forces memory copy
>>> c = torch.asarray(a, copy=True)
>>> a.data_ptr() == c.data_ptr()
False

>>> a = torch.tensor([1., 2., 3.], requires_grad=True)
>>> b = a + 2
```

```
>>> b
tensor([3., 4., 5.], grad_fn=<AddBackward0>)
>>> # Shares memory with tensor 'b', with no grad
>>> c = torch.asarray(b)
>>> c
tensor([3., 4., 5.])
>>> # Shares memory with tensor 'b', retaining autograd history
>>> d = torch.asarray(b, requires_grad=True)
>>> d
tensor([3., 4., 5.], grad_fn=<AddBackward0>)

>>> array = numpy.array([1, 2, 3])
>>> # Shares memory with array 'array'
>>> t1 = torch.asarray(array)
>>> array.__array_interface__['data'][0] == t1.data_ptr()
True
>>> # Copies memory due to dtype mismatch
>>> t2 = torch.asarray(array, dtype=torch.float32)
>>> array.__array_interface__['data'][0] == t2.data_ptr()
False

>>> scalar = numpy.float64(0.5)
>>> torch.asarray(scalar)
tensor(0.5000, dtype=torch.float64)
```

Notes & Warnings

NOTE

See also torch.tensor() creates a tensor that always copies the data from the input object. torch.from_numpy() creates a tensor that always shares memory from NumPy arrays. torch.frombuffer() creates a tensor that always shares memory from objects that implement the buffer protocol. torch.from_dlpack() creates a tensor that always shares memory from DLPack capsules.

Detailed Documentation

Documentation

Converts obj to a tensor.

a NumPy array or a NumPy scalar

an object that implements Python's buffer protocol

a sequence of scalars

When obj is a tensor, NumPy array, or DLPack capsule the returned tensor will, by default, not require a gradient, have the same datatype as obj, be on the same device, and share memory with it. These properties can be controlled with the dtype, device, copy, and requires_grad keyword arguments. If the returned tensor is of a different datatype, on a different device, or a copy is requested then it will not share its memory with obj. If requires_grad is True then the returned tensor will require a gradient, and if

obj is also a tensor with an autograd history then the returned tensor will have the same history.

When obj is not a tensor, NumPy array, or DLPack capsule but implements Python's buffer protocol then the buffer is interpreted as an array of bytes grouped according to the size of the datatype passed to the dtype keyword argument. (If no datatype is passed then the default floating point datatype is used, instead.) The returned tensor will have the specified datatype (or default floating point datatype if none is specified) and, by default, be on the CPU device and share memory with the buffer.

When obj is a NumPy scalar, the returned tensor will be a 0-dimensional tensor on the CPU and that doesn't share its memory (i.e. copy=True). By default datatype will be the PyTorch datatype corresponding to the NumPy's scalar's datatype.

When obj is none of the above but a scalar, or a sequence of scalars then the returned tensor will, by default, infer its datatype from the scalar values, be on the current default device, and not share its memory.

torch.tensor() creates a tensor that always copies the data from the input object. torch.from_numpy() creates a tensor that always shares memory from NumPy arrays. torch.frombuffer() creates a tensor that always shares memory from objects that implement the buffer protocol. torch.from_dlpack() creates a tensor that always shares memory from DLPack capsules.

obj (object) – a tensor, NumPy array, DLPack Capsule, object that implements Python's buffer protocol, scalar, or sequence of scalars.

dtype (torch.dtype, optional) – the datatype of the returned tensor. Default: None, which causes the datatype of the returned tensor to be inferred from obj.

copy (bool, optional) – controls whether the returned tensor shares memory with obj. Default: None, which causes the returned tensor to share memory with obj whenever

possible. If True then the returned tensor does not share its memory. If False then the returned tensor shares its memory with obj and an error is thrown if it cannot.

device (torch.device, optional) – the device of the returned tensor. Default: None, which causes the device of obj to be used. Or, if obj is a Python sequence, the current default device will be used.

requires_grad (bool, optional) – whether the returned tensor requires grad. Default: False, which causes the returned tensor not to require a gradient. If True, then the returned tensor will require a gradient, and if obj is also a tensor with an autograd history then the returned tensor will have the same history.
